

Compiling your own kernel

Kshitij Sobti

ksbitij.sobti@thinkdigit.com

This article will NOT tell you how exactly to configure your system for best performance, lowest memory or smallest size.

Instead we will tell you enough to get started and get exploring the Linux kernel.

A salute to the Kernel

Unless you are a hardcore Linux user, you might be a little overwhelmed by the subject of this article, but let us assure you, it's much easier than it sounds while being much more complicated than it looks!

For starters, let us give you an idea of what the kernel really is. In common usage, the "kernel" of something is its very essence, it is the idea at the very core. Quite appropriately, the Kernel of an operating system is its very core, the piece of code that interfaces between the computer's hardware and the applications running on it.

There are two basic approaches over the architecture of the kernel, on one side, there exists the "Monolithic Kernel", and on the other "Microkernel".

Monolithic kernels contain the entire OS within the kernel. This means all your driver and file-system code runs with highest privileges in the kernel's address-space.

In a microkernel-based OS, only the bare minimum code needed to run the system is part of the kernel, and the rest runs with lower privileges. This allows even system-level components such as drivers to be treated as applications, i.e. if it crashes it can simply be restarted, instead of requiring a system reboot.

It is possible to have a kernel which exhibits properties in between these two. The Windows NT kernel for example, which is used in Windows 2000 / XP / 2003 / Vista / 2008 / 7, is a hybrid kernel.

The Linux kernel itself is Monolithic,

however it is highly modular and supports loading such modules at runtime.

Different Kernel architectures

Monolithic kernel:



In a monolithic kernel, the kernel neatly separates the hardware devices and the applications which run on top of it. The drivers are "part" of the kernel as they reside in the same memory area.

Microkernel:

In a microkernel design the kernel manages a bare minimum of the system's hardware leaving the rest to device drivers which run in the user space.

The Linux Kernel

Technically, Linux IS the kernel, and "Linux" in itself is not a very usable OS. The many Linux distributions out there are actually software running on the Linux kernel.

Being a monolithic kernel, Linux is huge. The Linux kernel is a complicated piece of coding which — as of October 2010 — is over 13.5 million lines of code, spans over tens of thousands of files and supports dozens of platforms.

While the Linux kernel might not be a microkernel it is still quite modular. The Linux kernel can be — and has been — configured to run on an embedded device with lesser than an MB of ROM and RAM! The Linux kernel source code includes many switches that can be used to configure exactly what all features are built into the kernel, and how they are to be linked to the kernel. Additionally,

not all hardware that you would like your system to support needs to be built into the kernel, some of it can be built as external modules which can be loaded and unloaded by the kernel at run-time to

add support for certain features.

Even so, compiling this monster is a huge and complex task, but thankfully most of it is automated, and all you have to do is type a few commands, in fact configuring how the kernel should build can be done using a GUI.

Before we dig into that, we need to understand why there is any advantage in building your own kernel.

Why compile your own kernel anyway?

As we have said, compiling your own kernel is a huge task, which might be easy but is definitely not simple. So why do it in the first place? Why not just continue using the ready-made one which came with your distribution.

Nearly all Linux distributions out there include software, and a kernel that is compiled to run on the widest variety of hardware available. Even as of now, the 32-bit versions of most Linux distributions are capable of running on a Pentium or Pentium II! While you stand to gain by compiling any application for your hardware, you gain more from compiling system components such as KDE and Gnome and the kernel rather than a text editor like "gedit".

A lot of the hardware that is supported by your distribution's Kernel will never be of use to you, while you might be running